

Predavanje 2 — Paralelna računalna i arhitekture

Paralelno programiranje

Damir Krstinić Antonia Ivanda

FESB — Sveučilište u Splitu

1.

Modeli paralelnog računanja

Paralelno računalno

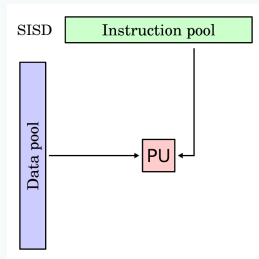
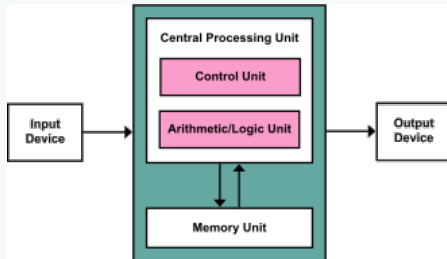
- ▶ Više procesorskih (računalnih) jedinica
 - ▶ više jezgri / procesora / računala
- ▶ Veza među računalnim jedinicama
 - ▶ sabirnica, mreža
- ▶ Arhitektura paralelnog računala
- ▶ Okruženje za razvoj i izvršavanje paralelnog algoritma
 - ▶ operativni sustav, razvojno okruženje
- ▶ Paralelni algoritam (i odgovarajući program pripremljen za arhitekturu koju koristimo)

Flynnova taksonomija (1966.)

S I S D Single Instruction Stream Single Data Stream	S I M D Single Instruction Stream Multiple Data Stream
M I S D Multiple Instruction Stream Single Data Stream	M I M D Multiple Instruction Stream Multiple Data Stream

Izvor: Wikipedia

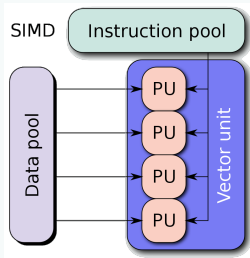
- ▶ **Von Neumannov model** računala (1946.)
- ▶ Jedan procesor, jedna memorijska jedinica
- ▶ Jedna instrukcija obrađuje jedan podatak
- ▶ Slijedno (serijsko) izvršavanje



Izvor: Wikipedia

Podatkovni paralelizam (*data-level parallelism*)

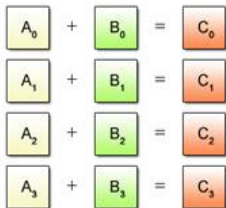
- ▶ Polje jednakih procesora
- ▶ Ista instrukcija izvodi se istovremeno na svim procesorima, nad **različitim** podacima
- ▶ Procesori rade sinkronizirano
- ▶ Primjena: vektorska obrada (jednolika); problem: grananje



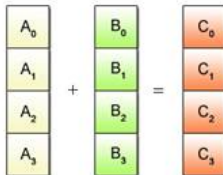
SIMD – izvedbe: array processor

- ▶ **Array processor** — vektorske (podatkovne) operacije
 - ▶ polje enakih jedinica izvršava istu operacijo nad različnimi elementi
 - ▶ ILLIAC IV (1966.); vektorska superračunala od 1970-ih

(a) Scalar Operation

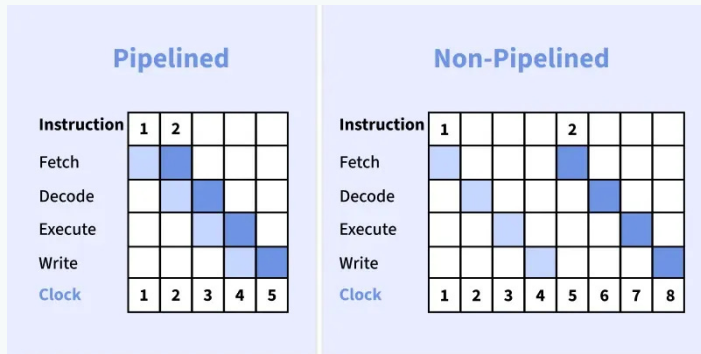


(b) SIMD Operation



SIMD – izvedbe: pipelined processor

- ▶ **Pipelined (protočni) processor** — register-based SIMD
 - ▶ prima jednu instrukciju, čita set podataka iz centralne memorije
 - ▶ obrada je podijeljena na faze koje se preklapaju kroz cikluse

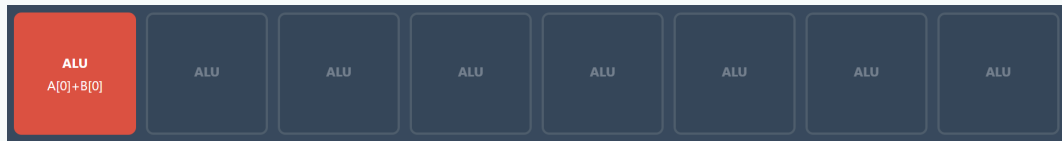


SIMD – izvedbe: asocijativni processor

- ▶ **Asocijativni processor** (*Associative / Content-Addressable Processor*) — podacima pristupa preko njihovog **sadržaja**, a ne preko memorijske adrese
- ▶ **Paralelno pretraživanje** — cijeli memorijski prostor pretražuje se u **jednom taktu**, umjesto „stavku po stavku”
- ▶ **Asocijativna memorija (CAM)** — *Content-Addressable Memory*; svaka ćelija ima ugrađenu logiku za usporedbu podataka
- ▶ **Prednosti:** brzina, masivni paralelizam
- ▶ **Nedostaci:** cijena, potrošnja energije, manji kapacitet memorije

SIMD – vektorski primjer

Skalarno (sekvencijalno) – 8 ciklusa: $A[i] + B[i] = C[i]$



SIMD (paralelno) – 1 ciklus: $A[0..7] + B[0..7] = C[0..7]$ – sve istovremeno!



SIMD divergencija

- Problem: **divergentno izvršavanje** (grananje)

```
forall (int i from 0 to 8) {  
    if (x[i] > 0)  
        y[i] = x[i] * x[i];    // MUL operacija  
    else  
        y[i] = x[i] + 100;    // ADD operacija  
}
```

- **Problem:** različiti ALU-ovi trebaju izvršiti **različite** instrukcije
- **Rješenje:** SIMD mora izvršiti **obje grane sekvencijalno**

SIMD – masking

SIMD MASKING - Timeline izvršavanja

ALU	Clock 1	Clock 2	Clock 3
ALU 0 ($x[0]>0$)	✓ MUL	X	✓
ALU 1 ($x[1]>0$)	✓ MUL	X	✓
ALU 2 ($x[2]>0$)	✓ MUL	X	✓
ALU 3 ($x[3]\leq 0$)	X	✓ ADD	✓
ALU 4 ($x[4]\leq 0$)	X	✓ ADD	✓
ALU 5 ($x[5]\leq 0$)	X	✓ ADD	✓
ALU 6 ($x[6]\leq 0$)	X	✓ ADD	✓
ALU 7 ($x[7]\leq 0$)	X	✓ ADD	✓
	IF grana	ELSE grana	nastavak



✓ Aktivno - rezultat se koristi



X Maskirano - rezultat se odbacuje

Performance Analiza

1

ciklus bez divergencije

2

ciklusa s divergencijom

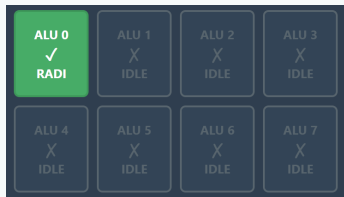
50%

peak performance

- ▶ 1 ciklus bez divergencije (optimalno)
- ▶ 2 ciklusa s divergencijom (**50% peak performance**)

SIMD – najgori slučaj

```
forall (int i from 0 to 8) {  
    if (i == 0) {  
        // 10 instrukcija, samo za prvi element!  
        y[i] = complex_calculation(x[i]);  
    }  
}
```



► Performance: **12,5 % = 1/8 peak**

Loše za SIMD

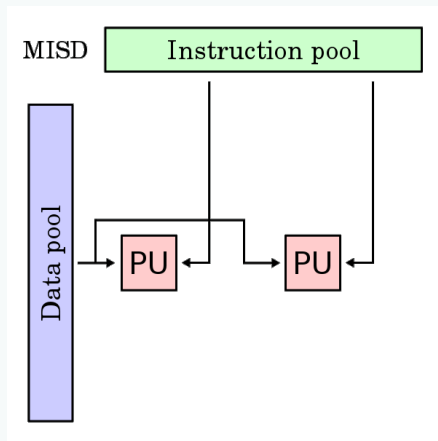
- ▶ puno IF/ELSE grananja
- ▶ nepravilne strukture (grafovi, stabla)
- ▶ neujednačeni podaci, različita grananja

Dobro za SIMD

- ▶ grafika (pikseli), matrice (jednolika obrada)
- ▶ audio obrada
- ▶ uniformni podaci, bez/malo grananja

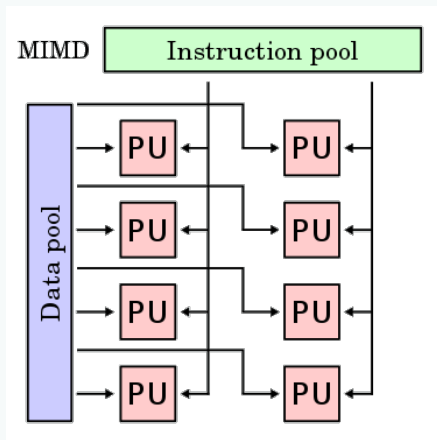
Paralelizam na razini operacije

- ▶ niz primitivnih operacija složenih u grupe koje se izvršavaju istovremeno nad **istim** podacima
- ▶ Primjena:
 - ▶ pipeline arhitekture
 - ▶ replikacija zadataka (*task replication*)
- ▶ Problem:
 - ▶ složena implementacija
- ▶ primjer: *Space Shuttle* flight control



Izvršavanje različitih operacija na različitim skupovima podataka

- ▶ više procesorskih jedinica izvršava **nezavisne** sljedove operacija na različitim podacima
- ▶ zadaćni paralelizam
- ▶ visoka fleksibilnost i efikasnost
- ▶ ključna komponenta modernih računalnih arhitektura



- ▶ Koristi se za širok spektar aplikacija (ne nužno HPC)
- ▶ U HPC-u često kao **manager/worker** arhitektura (jedan čvor dijeli poslove ostalima)
- ▶ Radi i s dijeljenom i s distribuiranom memorijom
- ▶ Prednosti: paralelno izvođenje više poslova, nezavisno izvršavanje
- ▶ Problemi: **load-balancing**, sinkronizacija, složenost algoritma

SPMD (*Single Program, Multiple Data*) — podvrsta MIMD arhitekture

- ▶ isti program izvršava se istovremeno na više procesora, nad **različitim** podacima
- ▶ procesori mogu komunicirati (npr. MPI ili dijeljena memorija)
- ▶ relativno jednostavniji kod (svi izvršavaju isti program)
- ▶ primjene: simulacije (klima, fluidi, financije), obrada slike, distribuirani sustavi

SPMD vs. SIMD

- ▶ **SIMD:** lock-step izvršavanje
 - ▶ bazirano na vektorskim procesorima
 - ▶ nemogućnost nezavisnog grananja
 - ▶ data-level parallelism
- ▶ **SPMD:** više čvorova izvršava isti program nezavisno, na različitim podacima
 - ▶ nezavisna grananja
 - ▶ data & task-level parallelism
 - ▶ fleksibilnost
 - ▶ komunikacija među čvorovima
 - ▶ složenija implementacija i upravljanje

2.

Paralelne procesorske arhitekture

Paralelizam na razini instrukcije (ILP) — simultano izvršavanje niza uputa unutar jednog programa

- ▶ jedna nit izvršavanja
- ▶ **statički** (software) paralelizam — prevoditelj u trenutku prevođenja optimira kod za paralelno izvršavanje
- ▶ **dinamički** (hardware) paralelizam — procesor u trenutku izvršavanja odlučuje koje se naredbe mogu izvršiti istovremeno
- ▶ instrukcije se izvršavaju paralelno na različitim izvršnim jedinicama unutar **jednog** CPU-a (ne na više jezgri!), npr. na različitim ALU/FPU
- ▶ dispečer čita upute i šalje svaku u jednu od izvršnih jedinica

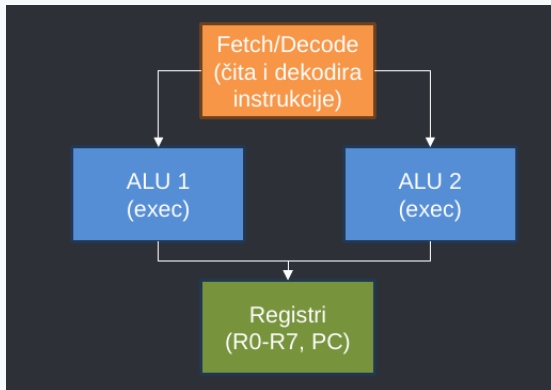
ILP — primjer

$a = b + c$

$d = e * f$

$g = a + d$

- ▶ **Clock 1:** ALU1 $a=b+c$, ALU2 $d=e*f$
- ▶ **Clock 2:** ALU1 $g=a+d$ (čeka)
- ▶ **2 ciklusa umjesto 3 → 1,5x brže**
- ▶ paralelizam unutar **jedne niti**



Superskalarni procesori

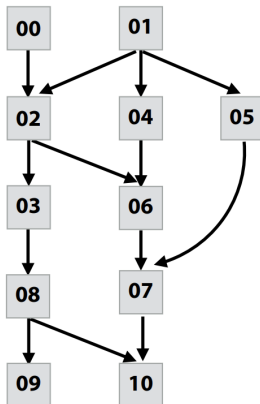
Program (sequence of instructions)

PC	Instruction	
00	a = 2	
01	b = 4	
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
10	else	
	print tmp7	

value during execution

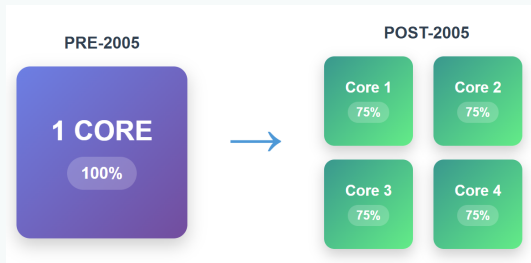


Instruction dependency graph



Višejezgreni (Multi-core) procesor

- ▶ **Prije 2005.:** jedan složen procesor · **poslije 2005.:** više jednostavnijih jezgri
- ▶ **Zašto?** *Power wall* — potrošnja raste s kvadratom frekvencije



Prije 2005.: jedan kompleksan procesor

- ▶ **Out-of-order execution** — izvršava instrukcije redom koji njemu odgovara, ne kako su napisane
- ▶ **Branch prediction** — pogađa grane IF-a i spekulativno ih izvršava unaprijed
- ▶ **Velike cache memorije** — skrivaju latenciju DRAM-a

Thread-level parallelism

- ▶ svaka jezgra ima: vlastiti PC, registre, L1 (32 KB) i L2 (256 KB) cache
- ▶ sve jezgre dijele: L3 cache (8-20 MB) i glavnu memoriju (DRAM)
- ▶ komunikacija među nitima preko dijeljene memorije
- ▶ alati: **OpenMP**, **pthread**s, **C++ std::thread**



Tri oblika paralelizma

- ▶ **ILP** — na razini instrukcije (automatski)
 - ▶ procesor sam pronalazi nezavisne instrukcije i izvršava ih paralelno na više izvršnih jedinica, bez intervencije programera
- ▶ **SIMD** — na razini podataka (vektorizacija)
 - ▶ jedna instrukcija istovremeno obrađuje više podataka, idealno za jednolike operacije nad poljima (vektori, matrice)
- ▶ **TLP** — na razini niti (eksplicitno)
 - ▶ više jezgri izvršava zasebne niti, a programer sam dijeli posao (OpenMP, pthreads)

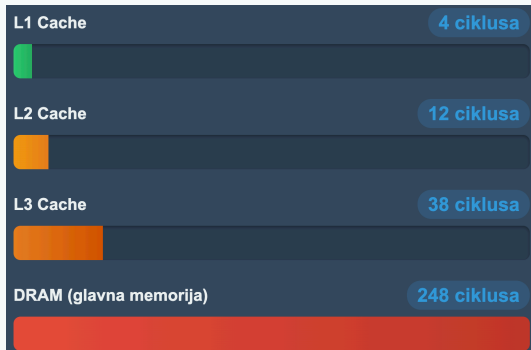
Moderni procesori kombiniraju sva tri!

3.

Cache memorija

Cache — memory latency

- Procesor većinu vremena provodi **čekajući** podatke iz memorije — vrijeme pristupa (Intel Kaby Lake, 4 GHz):



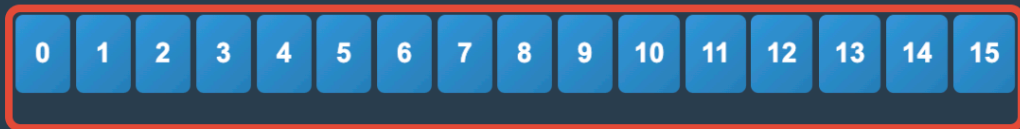
- **L1 cache** — najbrža, direktno na procesoru: 4 ciklusa (~ 1 ns)
- **L2 cache** — malo veća, malo dalja: 12 ciklusa (~ 3 ns)
- **L3 cache** — još veća, dijeljena među jezgrama: 38 ciklusa (~ 10 ns)
- **DRAM** — glavna memorija, gigabajti: 248 ciklusa (~ 62 ns)

- DRAM je $\sim 62\times$ **sporiji** od L1 cachea

Cache line

- ▶ Cache ne radi s pojedinačnim bajtovima, nego s **blokovima** (*cache line*)
- ▶ Cache line obično 64 B (16 int/float elemenata)

Memorija (array):



Cache locality

```
for (i = 0; i < N; i++)  
    sum += array[i];
```

*// array[i], array[i+1], array[i+2]...
// Sekvencijalno u memoriji!*

- ▶ **Prostorna lokalnost:** pristupiš li adresi X, vjerojatno ćeš uskoro i X+1, X+2... — npr. array[i], array[i+1] (susjedni podaci u memoriji)
- ▶ **Vremenska lokalnost:** pristupiš li podatku sada, vjerojatno ćeš mu ponovno uskoro — npr. sum u svakoj iteraciji

Dobar programer vodi računa o organizaciji cache memorije i lokalnosti

Cache — dobar vs. loš kod

```
// Dobro: pristup po recima (slijedno u memoriji)
```

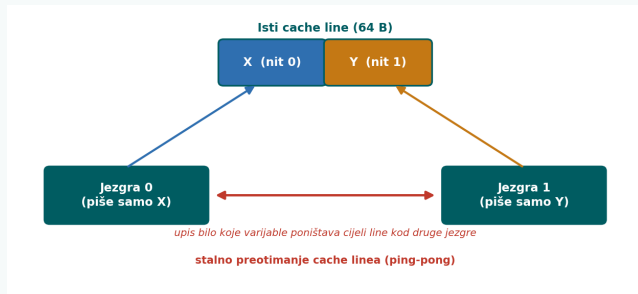
```
for (i = 0; i < N; i++)  
    for (j = 0; j < N; j++)  
        suma += A[i][j];
```

```
// Loše: pristup po stupcima (skokovi kroz memoriju)
```

```
for (j = 0; j < N; j++)  
    for (i = 0; i < N; i++)  
        suma += A[i][j];
```

- Isti algoritam, isti procesor — samo drukčiji uzorak pristupa memoriji → velika razlika u brzini

Lažno dijeljenje (*false sharing*)



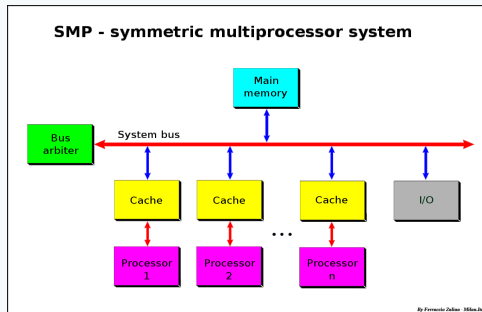
- ▶ Dvije niti mijenjaju **različite** varijable koje su slučajno u **istom cache lineu**
- ▶ Koherentnost radi na razini **cijelog linea**: svaki upis poništava line kod druge jezgre (**ping-pong**) → velik pad brzine iako logički nema dijeljenja
- ▶ Rješenje: **padding** / poravnanje — svaka varijabla u svom cache lineu

4.

Memorijske arhitekture

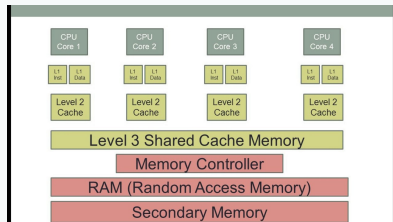
Dijeljena memorija

- ▶ Procesori izvršavaju nezavisne sljedove operacija, ali **dijele isti adresni prostor**
- ▶ *Bus-based*: svi procesori dijele istu sabirnicu



Izvor: Wikipedia

- ▶ *Non-Uniform Memory Access* — dijeljena memorija, svi procesori vide cijeli adresni prostor
- ▶ Brzina pristupa ovisi o **lokaciji** memorije u odnosu na procesor
- ▶ Složeno i skupo za implementaciju
- ▶ **ccNUMA** (cache-coherent) — osnova većine modernih višeprocorskih sustava

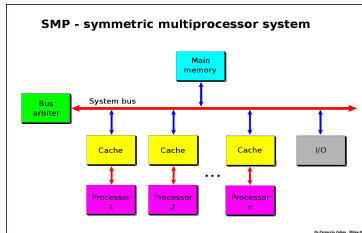


Dijeljena memorija — izazovi

- ▶ Programer mora **sinkronizirati** pristup memoriji
- ▶ **Zagušenje:** više procesora dijeli istu sabirnicu
- ▶ **Lažno dijeljenje:** sukob pri istovremenom pristupu istom bloku
- ▶ **Koherentnost cachea:** svaka promjena bloka mora se odraziti na sve kopije, inače procesori rade s nekoherentnim podacima

Symmetric Multiprocessing (SMP)

- ▶ Dva ili više **identičnih** procesora spojenih na zajedničku glavnu memoriju
- ▶ Nema procesora posebne namjene; dijele sve U/I uređaje
- ▶ Zajednički operativni sustav
- ▶ Većina višejezgrenih računala temelji se na SMP arhitekturi



Izvor: Wikipedia

- ▶ *Parallel Random Access Machine* — apstrakcija računala s dijeljenom memorijom
- ▶ Koristi se za **modeliranje** paralelnih algoritama i procjenu složenosti
- ▶ Modeli pristupa:
 - ▶ **EREW** — exclusive read, exclusive write
 - ▶ **CREW** — concurrent read, exclusive write
 - ▶ **CRCW** — concurrent read, concurrent write

Distribuirana memorija

- ▶ Svaki procesor (proces) ima **vlastiti** memorijski prostor
- ▶ Dijeljenje podataka **razmjenom poruka**
- ▶ Brz pristup lokalnim podacima, spor pristup udaljenima
- ▶ Zahtjevniji razvoj: podatke i strukture treba prilagoditi modelu
- ▶ Pri razmjeni poruka oba procesa (privremeno) zaustavljaju računanje

5.

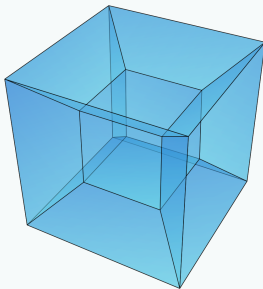
Mrežne arhitekture

Mrežne arhitekture

- ▶ **Potpuno povezane mreže:** svaki čvor povezan sa svima
 - ▶ složeno i skupo; složenost komunikacije $O(1)$
- ▶ **Ograničeno povezane mreže:** nema direktne veze svakog sa svakim
 - ▶ komunikacija se preusmjerava kroz druge čvorove
 - ▶ **linearni niz:** složenost mreže $O(N)$, komunikacije $O(N)$

Hypercube

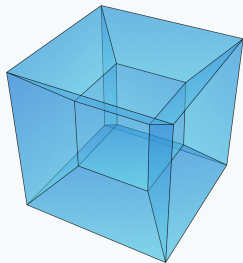
- ▶ N-dimenzionalna analogija kvadrata; procesori u vrhovima
- ▶ U sustavu s 2^N procesora svaki je povezan s **N** susjeda
- ▶ **Promjer mreže:** najmanji broj koraka do najudaljenijeg procesora $\rightarrow N$
- ▶ Broj procesora uvijek je potencija broja 2



Izvor: Wikipedia

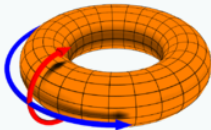
Mesh (rešetka)

- ▶ *Mesh Interconnection Network* — procesori u 2D rešetki
- ▶ Svaki procesor povezan sa **4** neposredna susjeda
- ▶ Broj procesora ne mora biti potencija broja 2
- ▶ Rubovi se mogu spojiti → **torus**



Torus

- ▶ Nastaje spajanjem rubova rešetke u zatvorenu petlju
- ▶ **1D torus (ring):** svaki čvor povezan s 2 susjeda
- ▶ **2D torus:** rešetka $n \times n$
- ▶ **3D torus:** svaki čvor povezan sa 6 susjeda
- ▶ **N-D torus:** svaki čvor povezan s $2N$ susjeda



Torus — prednosti i nedostaci

- ▶ **Prednosti:** veća brzina i manja latencija, bolja balansiranost, manja potrošnja energije
- ▶ **Nedostaci:** složenost, neujednačena dužina linkova, cijena
- ▶ Primjer: Fujitsu **6D torus** — 12× veća skalabilnost od 3D torusa
 - ▶ superračunalo **Fugaku** (~0,5 exaFLOPS, vrh TOP500 2020.-2022.)

6. **Sažetak**

Ključne poruke

- ▶ Flynnova taksonomija (SISD/SIMD/MISD/MIMD) klasificira arhitekture po tokovima instrukcija i podataka
- ▶ Moderni procesori kombiniraju **ILP, SIMD i TLP**, uz duboku **cache hijerarhiju**
- ▶ Dva memorijska modela: **dijeljena** (sinkronizacija, koherentnost) i **distribuirana** (razmjena poruka, topologije)

Što slijedi

- ▶ **Dio II** — dijeljena memorija: niti i OpenMP (Predavanja 4-7)
- ▶ **Dio III** — distribuirana memorija: MPI (Predavanja 8-9)
- ▶ **Dio IV** — GPU arhitekture: CUDA (Predavanja 10-11)